Formation IA - Brief de départ

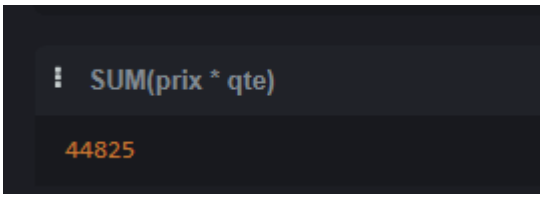
Question 3

Créez les requêtes SQL pour répondre aux questions clés sur les ventes de l'entreprise :

A. **Le chiffre d'affaires total**

Pour chaque ligne on doit calculer le CA de chaque vente pour ensuite faire une somme de toutes ces sommes.

SELECT SUM(prix * qte) FROM ventes ;

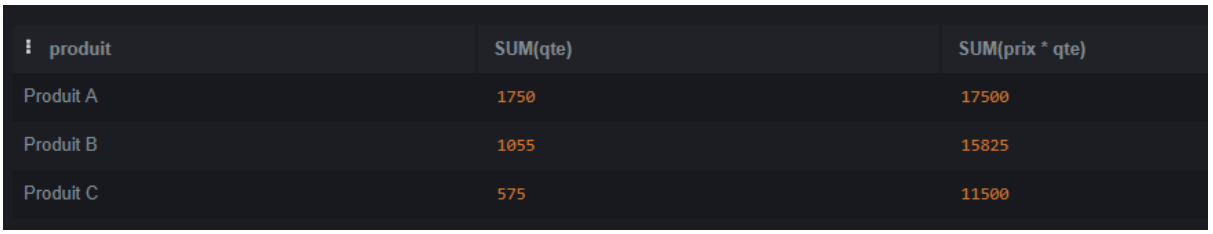


SUM(prix * qte)
44825

B. **Les ventes par produit**

On fait la somme de la colonne quantité et on rajoute un group by afin de calculer le CA par produit

SELECT produit, SUM(qte), SUM(prix * qte) FROM ventes GROUP BY produit;



produit	SUM(qte)	SUM(prix * qte)
Produit A	1750	17500
Produit B	1055	15825
Produit C	575	11500

C. **Les ventes par région**

De la même manière que la question précédente mais en utilisant le group by sur région

SELECT region, SUM(qte), SUM(prix * qte) FROM ventes GROUP BY region;



region	SUM(qte)	SUM(prix * qte)
Nord	1605	20725
Sud	1775	24100

Question 6

Tout d'abord on ajoute une colonne chiffre d'affaires (ca) afin de faciliter la suite de nos calculs.
Subtilité ici vis-à vis de SQL sur l'exercice précédent, le groupBy qu'on va mettre avant dans notre façon d'écrire les requêtes via le dataframe de panda.
Ensuite pour les demandes particulières, je me suis référé à la documentation de Pandas sur les dataframe (https://pandas.pydata.org/docs/reference/frame.html) où on retrouve les fonctions utiles a la résolution de notre problèmétique.

Pour la question 6 b-i/b-ii
j'ai eu l'occasion de tester la fonction agg(), qui permet d'agréger plusieurs opérations dans un seul appel de fonction.

```python
#6
print("Resultat question n°6")
data['ca'] = data['prix'] * data['qte']

print("CA par produit (CApp)")
print("Moyenne ")
CApp_moy = data.groupby('produit')['ca'].mean()
print("Resultat question n°6 a-i")
print("CApp_moy : " , CApp_moy)

print("Mediane")
CApp_med = data.groupby('produit')['ca'].median()
print("Resultat question n°6 a-ii")
print("CApp_med : " , CApp_med)


print("Volume de vente par produit (VVpp)")
print("Moyenne")
VVpp_moy = data.groupby('produit')['qte'].mean()
print("Resultat question n°6 a-i")
print("VVpp_moy : " , VVpp_moy)

print("Mediane")
VVpp_med = data.groupby('produit')['qte'].median()
print("Resultat question n°6 a-ii")
print("VVpp_med : " , VVpp_med)

print("pour le volume des ventes par produit")
# ecart-type > std
# variance > var
print("Resultat question n°6 b-ii")
VVpp_ET_VAR = data.groupby('produit')['qte'].agg(['std', 'var'])
print("VVpp_ET_VAR : " , VVpp_ET_VAR)
```

```
Resultat question n°6
CA par produit (CApp)
Moyenne
Resultat question n°6 a-i
CApp_moy :  produit
Produit A    1250.000000
Produit B    1217.307692
Produit C     958.333333
Name: ca, dtype: float64
Mediane
Resultat question n°6 a-ii
CApp_med :  produit
Produit A    1225.0
Produit B    1200.0
Produit C     900.0
Name: ca, dtype: float64
Volume de vente par produit (VVpp)
Moyenne
Resultat question n°6 a-i
VVpp_moy :  produit
Produit A    125.000000
Produit B     81.153846
Produit C     47.916667
Name: qte, dtype: float64
Mediane
Resultat question n°6 a-ii
VVpp_med :  produit
Produit A    122.5
Produit B     80.0
Produit C     45.0
Name: qte, dtype: float64
pour le volume des ventes par produit
Resultat question n°6 b-ii
VVpp_ET_VAR :                 std          var
produit
Produit A  38.078866  1450.000000
Produit B  21.228307   450.641026
Produit C  17.895953   320.265152
```

Question 7

Pour récupérer les ventes maximales et minimales en python j'ai transformé le dataframe en dictionnaire afin de parcourir chaque ligne et d'en récupérer les informations utiles à notre problème. Ici le nombre de ventes par produit.

On se retrouve ensuite avec un dictionnaire qui a en key le nom du produit et en valeur la somme des ventes de ce produit.

Python permet d'utiliser les fonctions min/max sur un dictionnaire afin de résoudre notre problématique

```python
# 7
print("Resultat question n°7")
data_python = data.to_dict(orient='records')

vpp = {}
for ligne in data_python:
    produit = ligne['produit']
    qte = ligne['qte']

    vpp[produit] = vpp.get(produit, 0) + qte

vente_max = max(vpp, key=vpp.get)
vente_min = min(vpp, key=vpp.get)

print(f"Plus vendu : {vente_max} ({vpp[vente_max]})")
print(f"Moins vendu : {vente_min} ({vpp[vente_min]})")
```

```
Resultat question n°7
Plus vendu : Produit A (1750)
Moins vendu : Produit C (575)
```

Question 8

Pour produire les graphiques demandés je me suis rendu sur la doc de plotly.
https://plotly.com/python/pie-charts/

j'ai du changer la forme de l'objet Series de Pandas via la fonction reset_index() afin de récupérer un dataframe avec deux colonnes exploitable par plotly

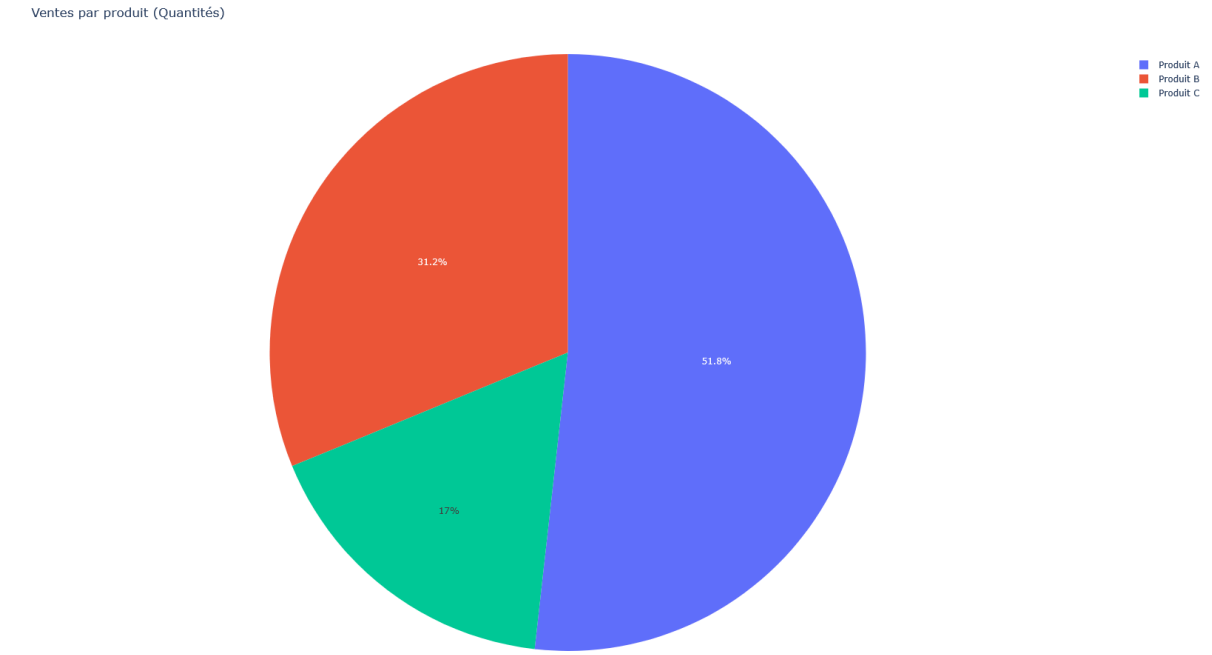
```
# 8.a
print("Resultat question n°8.a")
vpp_df=data.groupby('produit')['qte'].sum().reset_index()
figure_vpp_pie = px.pie(
    vpp_df,
    names='produit',
    values='qte',
    title="Ventes par produit (Quantités)"
)
figure_vpp_pie.write_html('vpp_pie.html')
print('Voir vpp_pie.html')

figure_vpp_bar = px.bar(
    vpp_df,
    x='produit',
    y='qte',
    title="Ventes par produit (Quantités)"
)
figure_vpp_bar.write_html('vpp_bar.html')
print('Voir vpp_bar.html')

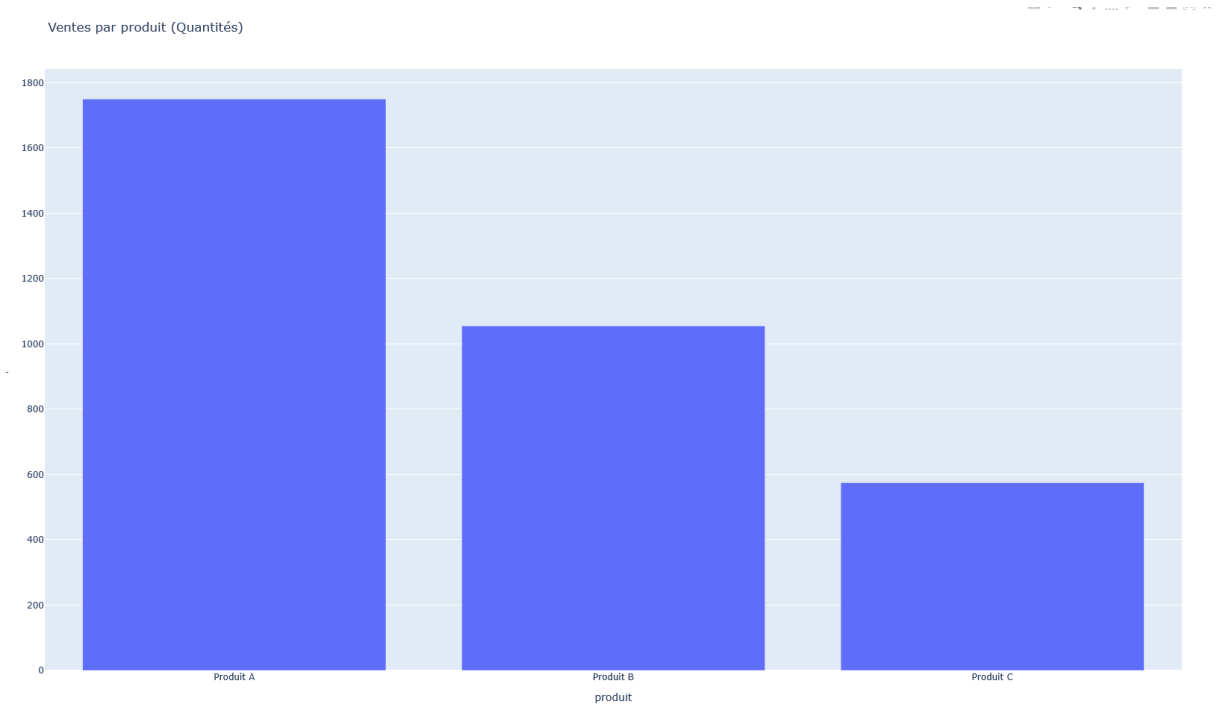
# 8.b
print("Resultat question n°8.b")
CAPP_df=data.groupby('produit')['ca'].sum().reset_index()
figure_vpp_pie = px.pie(
    CAPP_df,
    names='produit',
    values='ca',
    title="CA par produit (€)"
)
figure_vpp_pie.write_html('CAPP_pie.html')
print('Voir CAPP_pie.html')
```

8.a

Représentation dans un graphique en camembert via .pie()



Représentation dans un graphique en bar via .bar()



8.b

Représentation d’un graphique en camembert via .pie()

